SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	ARVIND .S	Reg. No.:	192525192
Section / Batch:		Date:	20/07/2026

Code of Conduct

I ARVIND.S *(Name / Reg No)*

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _____

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:		100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1). Doubly Linked List - Insertion and deletion.

1). Introduction :-

A Doubly Linked List (DLL) is a linked list in which each node contains three parts :-

1). Previous pointer (prev) - points to the previous node.

2). Data - stores the element.

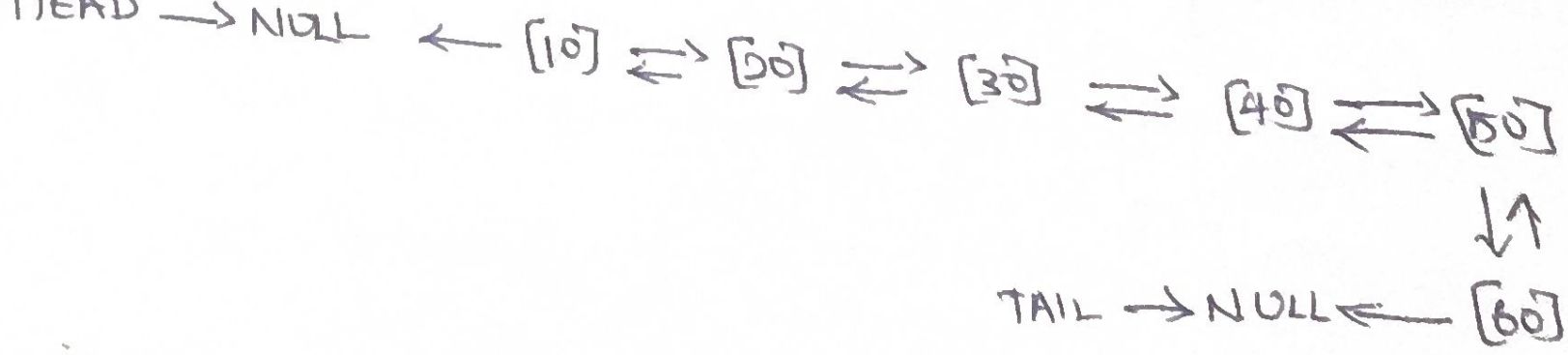
3). Next pointer (next) :- points to the next node

=> Structure of a node :-

Ex :- NULL ← [10] ⇄ [20] ⇄ [30] ⇄ [40] ⇄ [50] ⇄ [60] → NULL

(a) List contains 6 elements.

2) Initial Doubly Linked List



3). Insertion at 3rd position.

(a) Insert 25 at the 3rd position.

Before :-

After :-

pointer changed:

25 . prev = 20

25 . next = 30

20 . next = 25

30 . prev = 25

=> Delete the element at 2nd position.

(a) Delete 20 :-

Before :-

After :-

(a) pointer changed :-

10 . next = 25

25 . prev = 10 .

2. Stack Operations:-

Stack size = 5

Initial elements from 0 to 4:-

~~position:~~

Operation 1 :- Invert the stack

Position :-	0	1	2	3	4	
	88	66	33	55	22	← TOP

Operation 2 :- POP ()

Removes 22.

88 66 33 55 ← TOP

Operation 3 :- POP ()

88 66 33 ← TOP

Operation 4 :- PUSH (90)

88 66 33 90 ← TOP

Operation 5 :- PUSH (36)

88 66 33 90 36 ← TOP

Operation 6 :- PUSH (11)

Stack overflow because the stack is already full.

88 66 33 90 36 ← TOP

⊗ 11 is not inserted

Operation 7 :- PUSH (88)

Again ; stack overflow.

Operation 8 : POP ()

(P) Removes 36.

88 66 33 90 ← TOP

Operation 9 :- POP ()

(P) Removes 90

Final stack

position :- 0 1 2

88 66 33 ← TOP.